DATA MINING (CT-356)

Document-Based FAQ Chatbot

DEPARTMENT OF COMPUTERSCIENCE AND INFORMATION TECHNOLOGY (DATA-SCIENCE)

Submitted to :

Sir Rohail Qamar

Submitted by:

Fizza Ahmed DT-23304

M.Abdullah Kidwai DT-23302

Taaha Nadeem DT-23303

Wajiha Adnan DT-23001

Table of Contents

Abstract..... 3

Introduction.....4

1.1 What is a Chatbot?.....4
1.2 Why This Project is Needed.....4
1.3 Problem Statement.....4
1.4 Project Objectives.....5

Literature Review (2022-2025).....6

2.1 Modern AI Chatbots.....6
2.2 Retrieval Augmented Generation (RAG).....6
2.3 Vector Databases and FAISS.....6
2.4 LangChain as an Orchestration Framework.....7

Methodology.....8

3.1 Step 1 - Loading the Documents.....8
3.2 Step 2 - Splitting Text into Chunks.....8
3.3 Step 3 - Generating Embeddings.....8
3.4 Step 4 - Storing in FAISS.....8
3.5 Step 5 - Processing a User Query.....8
3.6 Step 6 - Retrieving Relevant Chunks.....8
3.7 Step 7 - Generating the Answer.....8
3.8 Step 8 - Displaying the Answer.....9
3.9 Data Processing and Mining Techniques.....9

System Design and Architecture.....10

4.1 preprocessing.py.....10
4.2 model.py.....10
4.3 chatbot.py.....10
4.4 app.py (Frontend).....10
4.5 How the Modules Connect.....10

Results and Discussion.....11

5.1 How Well Does the System Work?.....11
5.2 Output.....12
5.3 Advantages.....14
5.4 Limitations.....15

Conclusion.....16

Future Work.....17

References.....18

Abstract

Finding specific information in large document collections is slow and difficult. People often spend a lot of time reading PDFs, policies, emails, and chat records to find a single answer. This project solves that problem by building a chatbot that allows users to ask questions and get accurate answers instantly.

The system is a Document-Based FAQ Chatbot. It reads PDF and text files, understands their content, and stores it in a way that allows fast searching. When a user asks a question, it retrieves the most relevant parts of the documents and uses an AI language model to generate a clear answer.

The system is built using Python, LangChain, FAISS, HuggingFace Embeddings, and an OpenRouter GPT model. The interface is created with Streamlit, providing a clean and simple chat experience.

The final result is a chatbot that can handle multiple documents, answer questions accurately, and respond in a natural conversational way. It is fast, easy to use, and only provides answers based on the actual document content.

1. Introduction

1.1 What is a Chatbot?

A chatbot is a program that can have a conversation with a user by answering questions. It is used in websites, banks, and schools for support and guidance.

Early chatbots were rule-based and could only respond to fixed questions, failing when input changed. Modern AI chatbots use large language models trained on large amounts of text, allowing them to understand natural language and respond in a more human-like way.

1.2 Why This Project is Needed?

Many organisations such as companies, schools, hospitals, and government offices store large amounts of documents like policies, legal terms, emails, and support logs. Searching for specific information in them is slow and frustrating.

A better solution is a chatbot that understands these documents. Users ask a question, and the chatbot quickly returns the correct answer. This is what this project provides.

It has many uses: customer support teams can answer policy queries, students can search study material, and legal teams can find contract clauses. It is useful for anyone working with large text data.

1.3 Problem Statement

The core problem is that searching large document collections manually is inefficient. Keyword search often fails because it relies on exact words.

General AI chatbots cannot access private or organisation-specific documents and may use outdated or general information. A system is needed that uses specific documents and answers accurately without hallucinating.

1.4 Project Objectives

This project was built with the following clear goals:

- Build a system that reads and understands multiple documents including PDFs and text files.
- Convert document content into a searchable format using AI embeddings.
- Use FAISS to store and search through document content quickly.
- Connect a powerful language model to generate natural, clear answers.
- Display everything through a clean and simple Streamlit web interface.
- Ensure the system only answers from the provided documents and does not invent information.

2. Literature Review (2022-2025)

This section looks at what has happened in the world of AI chatbots and document-based question answering over the last three years. It helps us understand the background of this project and shows why the approach used here is a good one.

2.1 Modern AI Chatbots

Before 2022, most chatbots were simple rule-based systems with limited abilities. They could only respond to predefined inputs and were unable to understand context or handle follow-up questions.

From 2022 onwards, advanced Large Language Models (LLMs) were introduced by major companies. These models can understand and generate human-like language across many topics.

However, they still have a key limitation: they cannot access private or newly updated information and may produce incorrect answers, known as hallucinations.

2.2 Retrieval Augmented Generation (RAG)

RAG (Retrieval Augmented Generation) is a key idea in modern AI, introduced in 2020 and widely used from 2022 onwards.

Instead of relying only on memory, the system first retrieves relevant documents and then uses them as context for answering questions, similar to an open-book exam.

This improves accuracy and reduces hallucination because the model only uses provided information. RAG is now widely used for document-based chatbots, and this project is built on this approach.

2.3 Vector Databases and FAISS

To efficiently search documents, text is converted into numerical representations called vectors, which capture meaning.

A vector database stores these vectors and performs fast similarity searches to find relevant content.

FAISS is a popular open-source library for this purpose. It is fast, runs in memory, and works well for small to medium-sized datasets, making it suitable for this project.

2.4 LangChain as an Orchestration Framework

LangChain, introduced in 2022, is a Python library designed to simplify building applications with language models. It provides useful components such as document loaders, text splitters, vector store connectors, and prompt templates.

In 2023, LangChain introduced LCEL (LangChain Expression Language), which allows developers to define complete pipelines in a simple and structured way.

This project uses LCEL to connect the retriever, prompt, language model, and output parser into a single efficient workflow.

3. Methodology

This section explains exactly how the system works, step by step. It also explains the data processing and data mining techniques used and why each choice was made.

3.1 Step 1 - Loading the Documents

The system loads PDFs and text files using PyPDFLoader and TextLoader. In this project, four files are used: terms.pdf, privacy.pdf, emails.txt, and chat_logs.txt. All content is combined into document objects with text and metadata. This step is handled in the preprocessing module and supports multiple formats.

3.2 Step 2 - Splitting Text into Chunks

Text is split using RecursiveCharacterTextSplitter with 500-character chunks and 50-character overlap. The overlap preserves context between chunks, and natural boundaries are used where possible to improve readability.

3.3 Step 3 - Generating Embeddings

Each chunk is converted into embeddings using the all-MiniLM-L6-v2 model. These vectors represent meaning, allowing similar texts to be matched even with different wording. The model is fast, lightweight, and works offline.

3.4 Step 4 - Storing in FAISS

Embeddings are stored in a FAISS index for fast similarity search. The index is created at startup using build_vectorstore() and cached in Streamlit so it is reused for all queries.

3.5 Step 5 - Processing a User Query

User questions are passed into a LangChain pipeline that connects retrieval, prompt template, LLM, and output parser. The question is also converted into an embedding for comparison.

3.6 Step 6 - Retrieving Relevant Chunks

FAISS finds the top 5 most similar chunks (k=5). These are combined into a single context block containing the most relevant information for answering the question.

3.7 Step 7 - Generating the Answer

The context and question are passed into a ChatPromptTemplate with instructions to use only provided information and avoid hallucination. The prompt is sent to a GPT model via OpenRouter, and the response is returned as text.

3.8 Step 8 - Displaying the Answer

The final answer is shown in the Streamlit chat interface. Conversation history is stored for continuity, and a Clear Chat option resets the session.

3.9 Data Processing and Data Mining Techniques Used

This section summarises the core data processing and data mining techniques applied in the system:

- **Document Ingestion and Multi-Format Loading**

The system uses PyPDFLoader for PDFs and TextLoader for plain text files. This is a basic data ingestion technique used in NLP pipelines. Supporting multiple formats is important because real-world data comes in different file types.

- **Text Chunking with Overlap**

Text is split using RecursiveCharacterTextSplitter with 500-character chunks and 50-character overlap. Chunking is a segmentation technique, and overlap helps prevent loss of information between chunks, which is important in RAG systems.

- **Semantic Embedding (Feature Extraction)**

Text is converted into numerical vectors using the all-MiniLM-L6-v2 model. This feature extraction step captures semantic meaning, allowing similarity matching instead of simple keyword search

- **Vector Similarity Search (Information Retrieval)**

FAISS is used to perform k-NN search over embeddings, retrieving the top 5 most similar chunks (k=5). This is a standard nearest-neighbour information retrieval technique.

- **Prompt Engineering**

The system prompt includes rules to control the LLM's behaviour, such as avoiding hallucination and using only provided context. This ensures accurate and reliable responses.

- **LCEL Pipeline Composition**

angChain Expression Language connects retrieval, prompt creation, LLM, and output parsing into a single pipeline. This modular design makes the system clean, flexible, and easy to maintain.

4. System Design and Architecture

The system is built in a modular way, where each file has a specific role. This keeps the code clean, easy to maintain, and simple to understand. The system has four main modules:

4.1 preprocessing.py - The Data Preparation Module

This module loads PDFs (terms.pdf, privacy.pdf) and text files (emails.txt, chat_logs.txt) using appropriate loaders. All content is combined into a docs list, then split using RecursiveCharacterTextSplitter with 500-character chunks and 50-character overlap. This separation makes it easy to update data or chunk settings without affecting other parts of the system.

4.2 model.py - The Model Configuration Module

This module sets up the embedding model (all-MiniLM-L6-v2 via HuggingFaceEmbeddings) and the LLM (ChatOpenRouter with an API key stored in .env). It also includes build_vectorstore(), which creates the FAISS index from chunks. Centralising this makes model changes easy.

4.3 chatbot.py - The Question-Answering Logic Module

This module defines the system prompt and core logic. The prompt forces the model to use only provided context and avoid hallucination. The build_qa_chain() function creates a pipeline that retrieves the top 5 relevant chunks, formats them, and passes them to the LLM using LCEL for a single structured flow.

4.4 app.py - The Frontend and Session Manager

This is the Streamlit interface. It handles user input, display, and session state. A cached setup() function builds the FAISS index and QA chain at startup. The UI includes a dark theme, sidebar, chat interface, and a Clear Chat option. Conversation history is stored for continuity.

4.5 How the Modules Connect

app.py loads chunks from preprocessing.py, builds the vector store using model.py, and passes it to chatbot.py to create the QA chain. When a user asks a question, the chain retrieves relevant data, generates an answer, and returns it to the interface.

This clear separation makes the system modular, maintainable, and easy to extend.

5. Results and Discussion

This section discusses how the system performed during testing and provides an honest assessment of its strengths and limitations.

5.1 How Well Does the System Work?

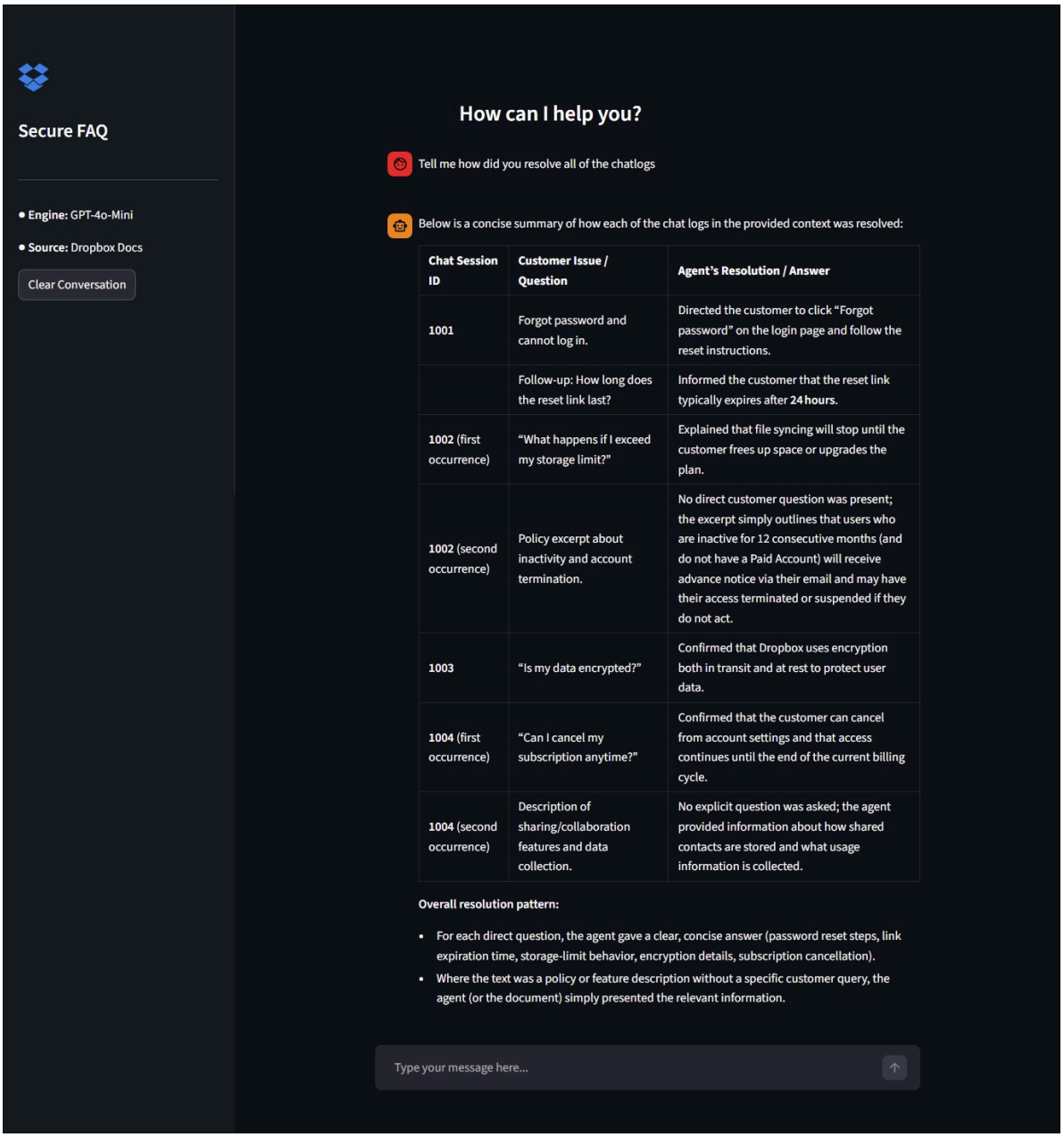
The chatbot was tested using four source documents: two PDFs (terms of service and privacy policy) and two text files (email logs and chat transcripts). A variety of questions were asked, including simple factual and complex multi-part queries.

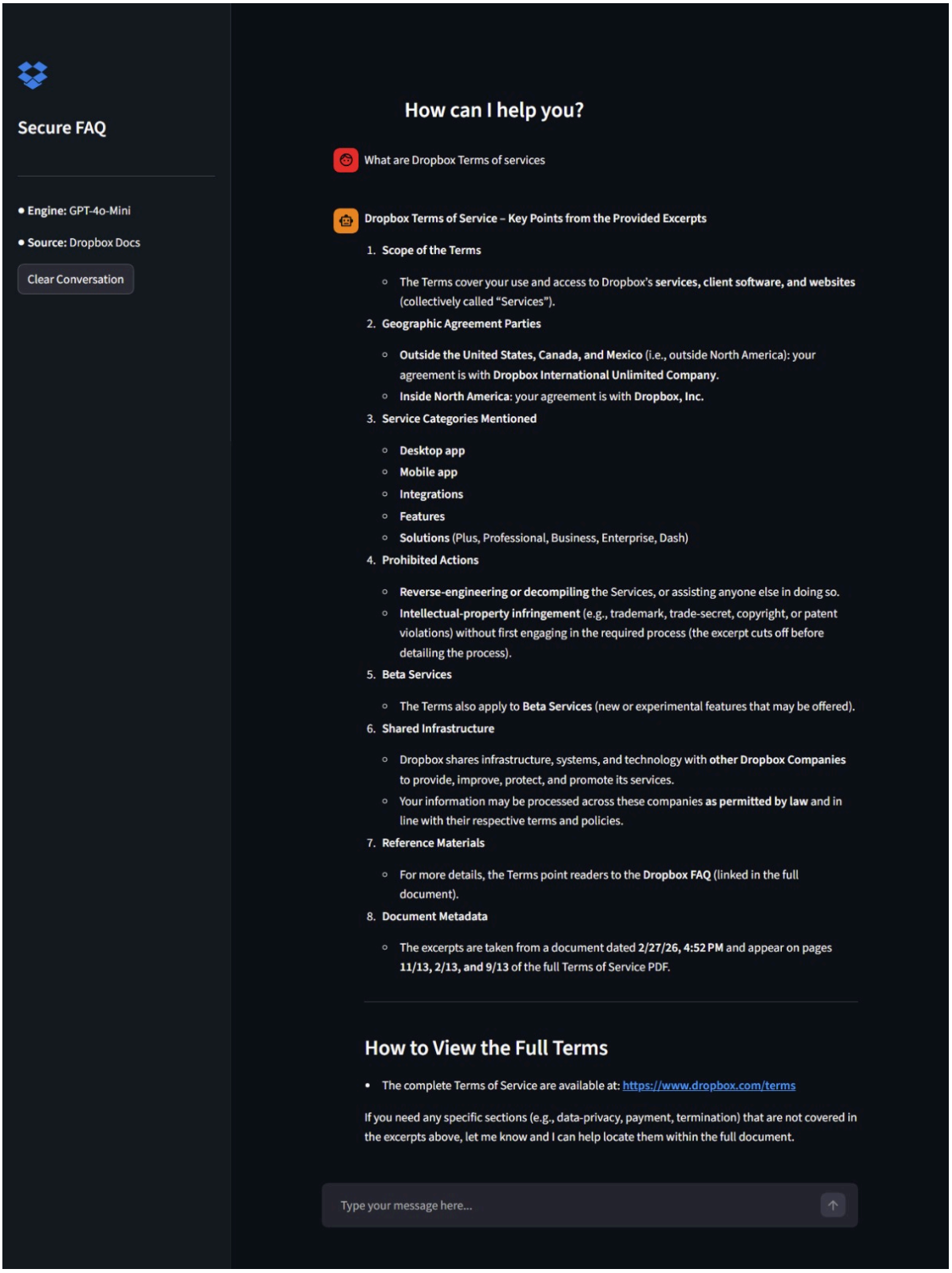
For direct factual questions, such as storage limits or data encryption, the system correctly retrieved relevant chunks and produced accurate answers. For broader questions, it combined multiple chunks and generated clear, structured responses, sometimes in table form.

For questions not covered in the documents, the system clearly stated that the information was not available, showing that it does not hallucinate or invent answers.

Response time was typically between two and five seconds per query, with most of the delay coming from the LLM response. FAISS retrieval is almost instant.

5.2 Output


Secure FAQ
Engine: GPT-4o-Mini
Source: Dropbox Docs
Clear Conversation
How can I help you?
Tell me how did you resolve all of the chatlogs
Below is a concise summary of how each of the chat logs in the provided context was resolved:
Chat Session ID | Customer Issue / Question | Agent's Resolution / Answer
1001 | Forgot password and cannot log in. | Directed the customer to click "Forgot password" on the login page and follow the reset instructions.
Follow-up: How long does the reset link last? | Informed the customer that the reset link typically expires after 24 hours.
1002 (first occurrence) | "What happens if I exceed my storage limit?" | Explained that file syncing will stop until the customer frees up space or upgrades the plan.
1002 (second occurrence) | Policy excerpt about inactivity and account termination. | No direct customer question was present; the excerpt simply outlines that users who are inactive for 12 consecutive months (and do not have a Paid Account) will receive advance notice via their email and may have their access terminated or suspended if they do not act.
1003 | "Is my data encrypted?" | Confirmed that Dropbox uses encryption both in transit and at rest to protect user data.
1004 (first occurrence) | "Can I cancel my subscription anytime?" | Confirmed that the customer can cancel from account settings and that access continues until the end of the current billing cycle.
1004 (second occurrence) | Description of sharing/collaboration features and data collection. | No explicit question was asked; the agent provided information about how shared contacts are stored and what usage information is collected.
Overall resolution pattern:
For each direct question, the agent gave a clear, concise answer (password reset steps, link expiration time, storage-limit behavior, encryption details, subscription cancellation).
Where the text was a policy or feature description without a specific customer query, the agent (or the document) simply presented the relevant information.
Type your message here...


Secure FAQ
Engine: GPT-4o-Mini
Source: Dropbox Docs
Clear Conversation
How can I help you?
What are Dropbox Terms of services
Dropbox Terms of Service – Key Points from the Provided Excerpts
1. Scope of the Terms
The Terms cover your use and access to Dropbox's services, client software, and websites (collectively called "Services").
2. Geographic Agreement Parties
Outside the United States, Canada, and Mexico (i.e., outside North America): your agreement is with Dropbox International Unlimited Company.
Inside North America: your agreement is with Dropbox, Inc.
3. Service Categories Mentioned
Desktop app
Mobile app
Integrations
Features
Solutions (Plus, Professional, Business, Enterprise, Dash)
4. Prohibited Actions
Reverse-engineering or decompiling the Services, or assisting anyone else in doing so.
Intellectual-property infringement (e.g., trademark, trade-secret, copyright, or patent violations) without first engaging in the required process (the excerpt cuts off before detailing the process).
5. Beta Services
The Terms also apply to Beta Services (new or experimental features that may be offered).
6. Shared Infrastructure
Dropbox shares infrastructure, systems, and technology with other Dropbox Companies to provide, improve, protect, and promote its services.
Your information may be processed across these companies as permitted by law and in line with their respective terms and policies.
7. Reference Materials
For more details, the Terms point readers to the Dropbox FAQ (linked in the full document).
8. Document Metadata
The excerpts are taken from a document dated 2/27/26, 4:52 PM and appear on pages 11/13, 2/13, and 9/13 of the full Terms of Service PDF.
How to View the Full Terms
The complete Terms of Service are available at: https://www.dropbox.com/terms
If you need any specific sections (e.g., data-privacy, payment, termination) that are not covered in the excerpts above, let me know and I can help locate them within the full document.
Type your message here...

5.3 Advantages

Advantage	Explanation
Simple to Use	The Streamlit interface requires no technical knowledge. Anyone can open the app, type a question, and read the answer.
Multiple Documents	The system reads and indexes all documents together. A question can be answered using information from any of the loaded files.
Semantic Search	The embedding-based approach finds relevant content by meaning, not just keywords. Different words with the same meaning are matched correctly.
Fast Retrieval	FAISS is highly optimised. The similarity search through thousands of chunks takes only milliseconds.
No Hallucination	The system prompt explicitly tells the AI not to make up information. It only answers from the retrieved context.
Session History	The full conversation is kept in memory during a session. Users can scroll back and review all previous answers.
Modular Code	Each part of the system is in its own file. Adding new features or changing one component does not break the others.

5.4 Limitations

Depends on Document Quality. If the source documents are poorly written, contain lots of errors, or are scanned as images rather than real text, the system may struggle to extract useful information. The quality of the answers is directly tied to the quality of the input documents.

Cannot Answer Outside the Documents. The chatbot only knows what is in the loaded files. If a user asks a general knowledge question or something not covered by any document, the system will say the information is not available. This is intentional but can sometimes be a limitation.

No Memory Between Sessions. When the browser is closed or the page is refreshed, the entire chat history is lost. There is no database storing previous conversations. Each session starts completely fresh.

Fixed Retrieval Depth (k=5). The system always retrieves exactly five chunks for every question. For very simple questions this might include some noise. For very complex questions it might not capture enough information.

Requires Internet for LLM. The chatbot needs a live internet connection to contact the OpenRouter API. The FAISS retrieval part works offline, but without internet the final answer cannot be generated.

FAISS Index Not Persistent. Every time the Streamlit server starts fresh, the FAISS index is rebuilt from scratch. For small document collections this is quick, but for very large collections it could take noticeable time.

6. Conclusion

This project set out to build a chatbot that reads documents and answers questions accurately, and this goal has been achieved.

The Document-Based FAQ Chatbot reads PDF and text files, splits them into chunks, converts them into embeddings, stores them in a FAISS index, and uses a GPT-class model to generate grounded answers. The system is provided through a simple Streamlit interface.

The RAG approach proved effective by retrieving relevant text before generating answers, which reduces hallucination. The system prompt also improves reliability by enforcing strict rules.

During testing, the system handled both simple and complex questions well and correctly refused to answer when information was not available in the documents.

The modular design (preprocessing, model setup, chatbot logic, and frontend) keeps the code clean and easy to extend.

Overall, the project shows how LangChain, FAISS, HuggingFace Embeddings, and OpenRouter LLMs can be combined to build a reliable document-based question-answering system with strong real-world potential.

7. Future Work

User Authentication and Access Control

Right now, anyone can use the application, which is not suitable for business use. A login system would enable private accounts and controlled access.Administrators could restrict access by roles (e.g., HR or legal), improving security and making the system more useful for organisations.

Persistent Chat History

Currently, conversations are lost when the session ends. In a future version, they could be saved in a database like SQLite or PostgreSQL.Users could revisit and search past conversations, making the chatbot more useful for long-term use.

Dynamic Document Upload

At the moment, documents are fixed at startup. A future version could allow users to upload documents through the web interface.The system would automatically process them, add them to the FAISS index, and make them searchable without restarting. This would make the system more flexible and self-managing.

Support for More Document Types

The current system only supports PDFs and text files. Future versions could add Word, Excel, PowerPoint, web pages, and audio transcripts.Supporting more formats would make the system more useful across different industries.

Better Embedding and Retrieval Models

The system uses all-MiniLM-L6-v2, which is suitable for this scale, but stronger embedding models like BGE-large could improve accuracy.Also, hybrid retrieval (combining vector search with keyword search) could produce better results than using either method alone.

Cloud Deployment

Currently the system runs locally. To support multiple users, it could be deployed on the cloud using platforms like AWS, Google Cloud, Azure, or HuggingFace Spaces.Using Docker would package the application and make deployment easier and reproducible across different environments.

References

1. Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. Advances in Neural Information Processing Systems (NeurIPS 2020).
2. Gao, Y., Xiong, Y., Gao, X., Jia, K., Pan, J., Bi, Y., et al. (2023). Retrieval-Augmented Generation for Large Language Models: A Survey. arXiv preprint arXiv:2312.10997.
3. Reimers, N. and Gurevych, I. (2019). Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. Proceedings of EMNLP 2019. Association for Computational Linguistics.
4. Johnson, J., Douze, M. and Jegou, H. (2019). Billion-scale similarity search with GPUs. IEEE Transactions on Big Data, 7(3), pp. 535-547.
5. Chase, H. (2022). LangChain: Building Applications with LLMs through Composability. Available at: https://github.com/langchain-ai/langchain
6. Streamlit Inc. (2022-2024). Streamlit Documentation - Build and Share Data Apps. Available at: https://docs.streamlit.io
7. OpenRouter. (2023-2024). OpenRouter API Documentation. Available at: https://openrouter.ai/docs
8. HuggingFace. (2022-2024). all-MiniLM-L6-v2 Model Card. Available at: https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2
9. Shi, F., Chen, X., Misra, K., Scales, N., Dohan, D., Chi, E., et al. (2023). Large Language Models Can Be Easily Distracted by Irrelevant Context. Proceedings of ICML 2023.